

R-Type Instructions

Name	Instruction Syntax	Description	Example
add	<code>add rd, rs1, rs2</code>	$rd = rs1 + rs2$	add x18, x20, x21. [x20 register's data added with x21's data and stored in x18 register.]
subtract	<code>sub rd, rs1, rs2</code>	$rd = rs1 - rs2$	sub x18, x20, x21. [x21's data is subtracted from x20 and stored in x18 register.]
Bitwise and	<code>and rd, rs1, rs2</code>	$rd = rs1 \& rs2$	and x5, x6, x7. [bitwise and operation between x6 and x7 register and saved in x5.]
Bitwise or	<code>or rd, rs1, rs2</code>	$rd = rs1 rs2$	or x5, x6, x7. [bitwise or operation between x6 and x7 register and saved in x5.]
Bitwise xor	<code>xor rd, rs1, rs2</code>	$rd = rs1 \wedge rs2$	xor x5, x6, x7. [bitwise xor operation between x6 and x7 register and saved in x5.]
Shift left logical	<code>sll rd, rs1, rs2</code>	$rd = rs1 \ll rs2$	sll x20, x21, x22. [x21's data is shifted left by x22 and saved in x20.] <i>The number of shifting times depends on x22. If x22 = 5, then x21 will be shifted left by 5 times.</i>
Shift right logical	<code>srl rd, rs1, rs2</code>	$rd = rs1 \gg rs2$	srl x20, x21, x22. [x21's data is shifted right by x22 and saved in x20.] <i>The number of shifting times depends on x22. If x22 = 3, then x21 will be shifted right by 3 times.</i>
Multiply	<code>mul rd, rs1, rs2</code>	$rd = rs1 * rs2$	mul x20, x21, x22. [x21's data is multiplied with x22's data and only the lower 64 bits of the product are stored in x20]
Multiply high	<code>mulh rd, rs1, rs2</code>	$rd = rs1 * rs2$	mulh x20, x21, x22. [x21's data is multiplied with x22's data and only the upper 64 bits of the product are stored in x20.] Both rs1 and rs2 are signed

Multiply high unsigned	<code>mulhu rd, rs1, rs2</code>	$rd = rs1 * rs2$	mulhu x20, x21, x22. [x21's data is multiplied with x22's data, and only the upper 64 bits of the product is stored in x20] Both rs1 and rs2 are unsigned
Multiply high signed/unsigned	<code>mulhsu rd, rs1, rs2</code>	$rd = rs1 * rs2$	mulhsu x20, x21, x22. [x21's data is multiplied with x22's data, and only the upper 64 bits of the product is stored in x20.] rs1 is signed, and rs2 is unsigned.

***NOTE:** *rs1 = source register 1, rs2 = source register 2, rd = destination register*
All multiply instructions can only be used after the midterm.

I-Type Instructions

Name	Instruction Syntax	Description	Example
add immediate	<code>addi rd, rs, imm</code>	$rd = rs1 + imm$	addi x18, x20, 5. [x20 register's data added with constant 5 and stored in x18 register.] If subtraction is needed, then 2's complement of immediate is used for the negative number.
Bitwise and immediate	<code>andi rd, rs, imm</code>	$rd = rs1 \& imm$	andi x5, x6, 45. [bitwise and operation between x6 and constant 45 and saved in x5]
Bitwise or immediate	<code>ori rd, rs, imm</code>	$rd = rs1 imm$	ori x5, x6, 22. [bitwise or operation between x6 and constant 22 and saved in x5.]
Bitwise xor immediate	<code>xori rd, rs, imm</code>	$rd = rs1 \wedge imm$	xor x5, x6, 12. [bitwise xor operation between x6 and constant 12 and saved in x5.]
Shift left logical immediate	<code>slli rd, rs, imm</code>	$rd = rs1 \ll imm$	slli x20, x21, 4. [x21's data is shifted left by 4 times and saved in x20.]
Shift right logical immediate	<code>srli rd, rs, imm</code>	$rd = rs1 \gg imm$	srli x20, x21, 3. [x21's data is shifted right by 3 times and saved in x2.]

Memory Instructions (LOAD) [Retrieve data from RAM to Register]			
Load byte	<code>lb rd, imm(rs1)</code>	rd = M[rs1 + imm] [7:0] → 8 bits (immediate is sign extended)	lb x20, 1(x25). [Signed byte is loaded from memory to x20 register]. Each index of array is stored using 1 memory slot/location
Load byte unsigned	<code>lbu rd, imm(rs1)</code>	rd = M[rs1 + imm] [7:0] → 8 bits (immediate is extended with zero)	lbu x20, 1(x25). [Unsigned byte is loaded from memory to x20 register]. Each index of array is stored using 1 memory slot/location
Load half word	<code>lh rd, imm(rs1)</code>	rd = M[rs1 + imm] [15:0] → 16 bits (immediate is sign extended)	lh x20, 2(x25). [Signed half word is loaded from memory to x20 register]. Each index of array is stored using 2 memory slot/location
Load half word unsigned	<code>lhu rd, imm(rs1)</code>	rd = M[rs1 + imm] [15:0] → 16 bits (immediate is extended with zero)	lhu x20, 2(x25). [Unsigned half word is loaded from memory to x20 register]. Each index of array is stored using 2 memory slot/location
Load word	<code>lw rd, imm(rs1)</code>	rd = M[rs1 + imm] [31:0] → 32 bits (immediate is sign extended)	lw x20, 4(x25). [Signed word is loaded from memory to x20 register]. Each index of array is stored using 4 memory slot/location
Load word unsigned	<code>lwu rd, imm(rs1)</code>	rd = M[rs1 + imm] [31:0] → 32 bits (immediate is extended with zero)	lwu x20, 4(x25). [Unsigned word is loaded from memory to x20 register]. Each index of array is stored using 4 memory slot/location
Load double word	<code>ld rd, imm(rs1)</code>	rd = M[rs1 + imm] [63:0] → 64 bits (immediate is sign extended)	ld x20, 16(x25). [Signed double word is loaded from memory to x20 register]. Each index of array is stored using 8 memory slot/location
Load double word unsigned	<code>ldu rd, imm(rs1)</code>	rd = M[rs1 + imm] [63:0] → 64 bits (immediate is extended with zero)	ldu x20, 16(x25). [Unsigned double word is loaded from memory to x20 register]. Each index of array is stored using 8 memory slot/location
JALR			
Jump and Link Register	<code>jalr rd, imm(rs1)</code>	PC = rs1 + imm	jalr x0, 0(x1). Returns control to the caller

***NOTE:** $M[rs1 + imm]$ means data is loaded from memory(RAM). The target address is calculated as follows: **Target address = $rs1 + imm$** . Here, $rs1$ represents the **base register** and immediate represents the **offset**.

Each memory slot/location stores 8 bits. **[RISC V is byte addressed]**

- So 1 double word requires = $64/8 = 8$ slots/locations of memory
- So 1 word requires = $32/8 = 4$ slots/locations of memory
- So 1 half word requires = $16/8 = 2$ slots/locations of memory
- So 1 byte requires = $8/8 = 1$ slot/location of memory

S - Type Instructions

Memory Instructions (STORE) [Store data from Register to RAM]			
Name	Instruction Syntax	Description	Example
Store byte	<code>sb rs2, imm(rs1)</code>	$M[rs1 + imm] = rs2[7:0] \rightarrow 8$ bits (immediate is sign extended)	sb x20, 1(x25) . [Signed byte is stored from x20 register to memory]. Each index of array is stored using 1 memory slot/location
Store half word	<code>sh rs2, imm(rs1)</code>	$M[rs1 + imm] = rs2[15:0] \rightarrow 16$ bits (immediate is sign extended)	sh x20, 2(x25) . [Signed half word is stored from x20 register to memory]. Each index of array is stored using 2 memory slot/location
Store word	<code>sw rs2, imm(rs1)</code>	$M[rs1 + imm] = rs2[31:0] \rightarrow 32$ bits (immediate is sign extended)	sw x20, 4(x25) . [Signed word is stored from x20 register to memory]. Each index of array is stored using 4 memory slot/location
Store double word	<code>sd rs2, imm(rs1)</code>	$M[rs1 + imm] = rs2[63:0] \rightarrow 64$ bits (immediate is sign extended)	sd x20, 16(x25) . [Signed double word is stored from x20 register to memory]. Each index of array is stored using 8 memory slot/location

*** RISC V does not have unsigned store instructions. Only Load has unsigned.**

SB - Type Instruction [Conditional Branching]

Name	Instruction Syntax	Description	Example
Branch if equal	<code>beq rs1, rs2, label</code>	If $rs1 == rs2$: PC = PC + offset *jump to label Else: PC = PC + 4	beq x5, x7, exit. [If x5 value and x7 value are equal, then jump to the exit label.]
Branch if not equal	<code>bne rs1, rs2, label</code>	If $rs1 != rs2$: PC = PC + offset *jump to label Else: PC = PC + 4	bne x5, x7, exit. [If x5 value and x7 value are not equal, then jump to the exit label.]
Branch if less than (signed)	<code>blt rs1, rs2, label</code>	If $rs1 < rs2$: PC = PC + offset *jump to label Else: PC = PC + 4	blt x20, x22, IF. [If x20 value is less than x22 value, then jump to the IF label] x20 and x22 are signed values.
Branch if less than (unsigned)	<code>bltu rs1, rs2, label</code>		bltu x20, x22, IF. [If x20 value is less than x22 value, then jump to the IF label] x20 and x22 are unsigned values
Branch if greater or equal (signed)	<code>bge rs1, rs2, label</code>	If $rs1 \geq rs2$: PC = PC + offset *jump to label Else: PC = PC + 4	bge x25, x26, ELSE. [If x25 value is greater or equal to x26, then jump to ELSE label] x25 and x26 are signed values
Branch if greater or equal (unsigned)	<code>bgeu rs1, rs2, label</code>		bgeu x25, x26, ELSE. [If x25 value is greater or equal to x26, then jump to ELSE label] x25 and x26 are unsigned values

**NOTE: label can be any placeholder name that defines a specific memory address.*

Naming convention: Do not start with any numbers or special characters.

UJ - Type Instruction [Unconditional Jump]

Name	Instruction Syntax	Description	Example
Jump and Link	<code>jal rd, procedureLabel</code>	$rd = PC + 4$ $PC = PC + \text{offset}$	<code>jal x1, functionA</code> . Jumps to the function block. Saves the return address in x1.

U - Type Instruction [Upper immediate]

Name	Instruction Syntax	Description	Example
Load upper immediate	<code>lui rd, immediate</code>	$rd[31:12] = \text{imm}$	<code>lui x20, 976</code> . copies the 20-bit immediate to the upper 20 bits of x20[31:12]

References:

[Cornell](#)

[RISC V](#)